



SENAI

1º Pilar

Encapsulamento

Definição

É um princípio que consiste em esconder detalhes de implementação de um componente, expondo apenas operações seguras e que o mantenha em um estado consistente.

Analógia



Regra de ouro do encapsulamento

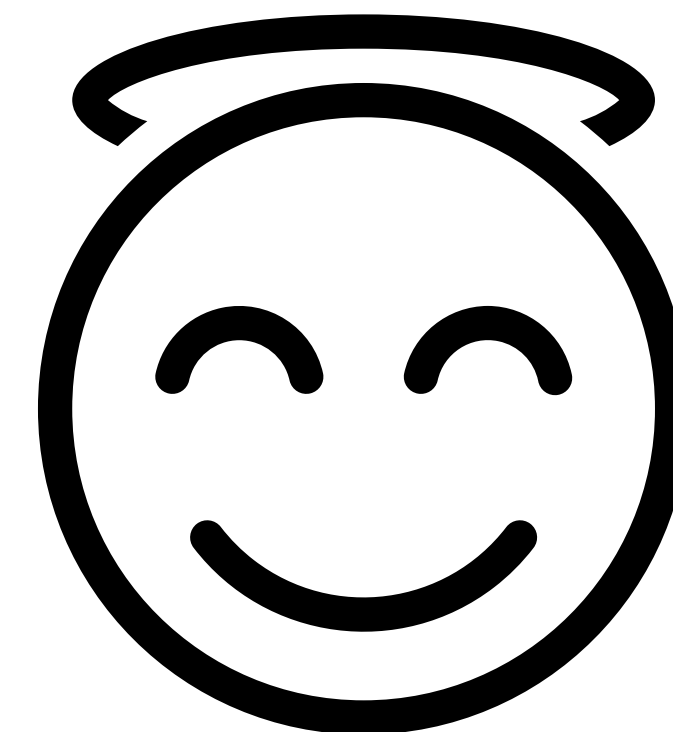
O objeto deve sempre estar em um estado consistente, é uma prática fundamental em programação orientada a objetos (OOP) para garantir a integridade dos dados e o comportamento esperado da aplicação e a própria classe deve garantir isso.

Tabela dos modificadores de acesso

	Própria classe	Subclasses no assembly	Classes do assembly	Subclasses fora do assembly	Classes fora do assembly
Public	X	X	X	X	X
Protected	X	X		X	
Private	X				

Porém, em Python não existem esses modificadores!

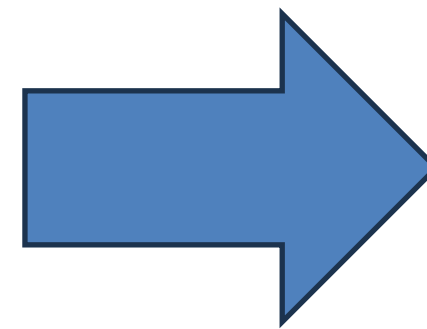
Somos “Adultos responsáveis”



Encapsulamento – Implementação manual

Todo atributo é definido como privado quando colocado o caractere underline duas vezes em sequência;

```
class Produto:
    nome: str
    preco: float
    quantidade: int
```



```
class Produto:
    __nome: str
    __preco: float
    __quantidade: int
```

Assim, se implementa-se métodos ou propriedades para acessar os membros privados da classe

Properties

Properties = Propriedades

Uma propriedade é um membro que oferece um mecanismo flexível para ler, gravar ou calcular o valor de um campo particular.

As propriedades podem ser usadas como se fossem membros de dados públicos, mas são métodos realmente especiais chamados acessadores.

Esse recurso permite que os dados sejam acessados facilmente e ainda ajuda a promover a segurança e a flexibilidade dos métodos.

Propriedades em python

Você pode usar o decorador **@property** para criar o método **@nome.getter** (para obter o valor), **@nome.setter** para o método setter (para definir o valor), e **@nome.deleter** para o método deleter (para remover o valor). Isso é útil para encapsular a lógica de negócio, adicionar validações, ou transformar um atributo público em um atributo privado com acesso controlado.

Propriedades em python

```
#Propriedades
...
@property
def nome(self) -> str:
    return self.__nome
```

```
# Métodos Setters, Getters e Deleters
...
@nome.setter
def nome(self, nome: str):
    self.__nome = nome
```

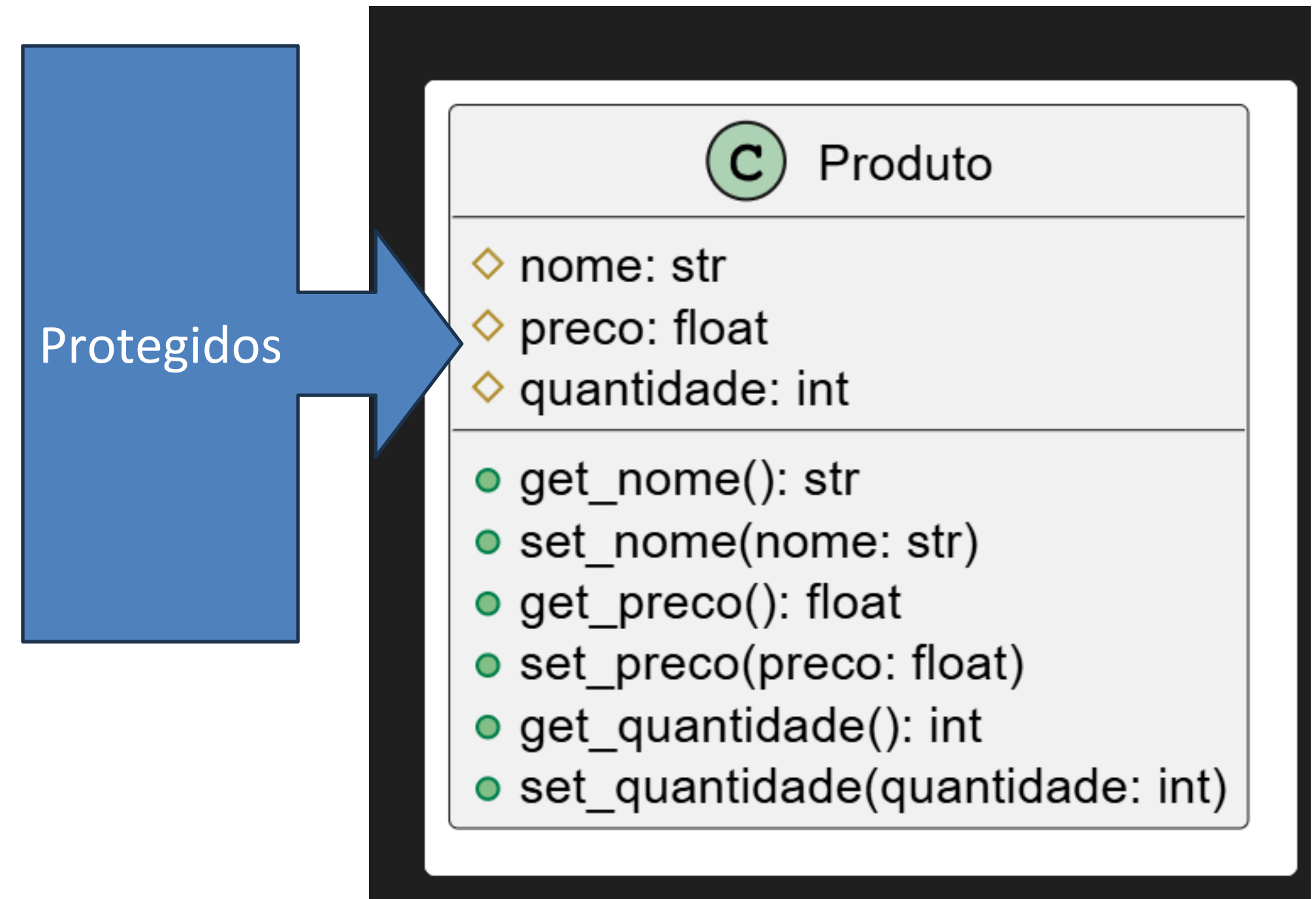
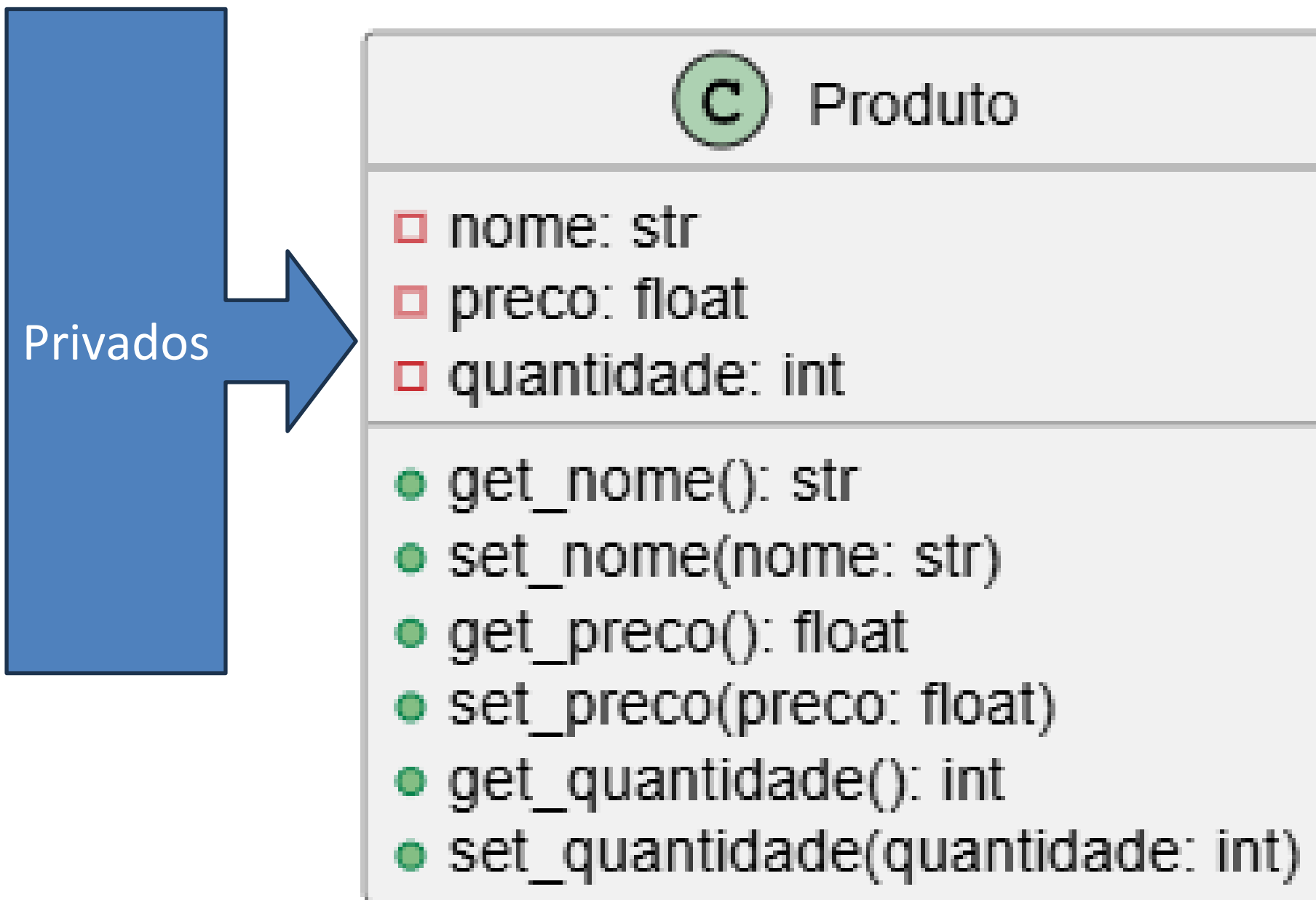
```
@nome.getter
def nome(self) -> str:
    return self.__nome
```

```
@nome.deleter
def nome(self):
    del self.__nome
```

Projeto UML

da classe

Projeto UML da classe



Sugestão para implementação de membros

1. Atributos privados;
2. Propriedades autoimplementadas;
3. Construtores;
4. Propriedades customizadas;
5. Outros métodos da classe;

Exercícios



Google Classroom



A nighttime photograph of a city street, likely in São Paulo, featuring light trails from moving vehicles and illuminated buildings in the background. The image is used as a background for a promotional graphic.

SENAI

DEPARTAMENTO REGIONAL
DE SÃO PAULO

www.sp.senai.br